
F21SC- INDUSTRIAL PROGRAMMING

C# web browser

ARCHANA THARAMMAL

MSC DATA SCIENCE

H00488368

TABLE CONTENT

S.No	Content	Page no
1	Introduction	2
2	Requirements checklist	3
3	Design considerations	4
4	User Guide	5
5	Developer Guide	6
6	Testing	7
7	Reflections on programming language and implementation	8
8	Conclusion	9
9	Reference	10

INTRODUCTION

This is a simple, custom-built web browser application, created using C# and Windows Forms. The primary purpose is to provide a browser that can support essential browsing capabilities, such as sending HTTP request messages for URLs typed by user, webpage navigation, managing navigation history, handling favourites, and supporting bulk download functionality and others. This application further demonstrates essential concepts in file management, and responsive user interface design without relying on the 'WebBrowser' class, allowing to go much deeper into manual handling of HTTP requests and responses.

During the development process, several assumptions were made to streamline functionality while focusing on core requirements. First, the program assumes that user-entered URLs are valid and prefixed correctly with "http://" or "https://"; if not, the application automatically verifies and corrects the prefix. Additionally, to enhance user experience, the design offer intuitive keyboard shortcuts for navigation. For example, hitting enter to load a URL, Ctrl+B and Ctrl+F for backward and forward navigation, and Ctrl+R for refreshing the page. Lastly, for favourites and history management, it is the storage files ('favourites.txt' and 'history.txt') are accessible in the application directory and available for reading and writing at runtime.

This report shows the browser's functional components, user interactions, and data handling mechanisms, describing how each aspect was designed to contribute to an efficient, user-friendly browsing experience.

REQUIREMENTS CHECKLIST

List of requirements implemented

- Sending HTTP request messages for URLs typed by the user– The browser allows users to enter URLs, sending HTTP requests for each input.
- Receiving HTTP response messages and displaying HTML code– The application displays the raw HTML content of the received HTTP response within the interface.
- Display HTTP response status code and webpage title– The application shows the HTTP response status code and title of the webpage in the main window.
- HTTP response status codes handling:
 - 200 OK– The application successfully handles and displays the 200 OK status for valid URLs.
 - 400 Bad Request–Correctly displays error messages for 400 Bad Request responses.
 - 403 Forbidden- Handles and displays the 403 Forbidden error message as required.
 - 404 Not Found– Displays the 404 Not Found error message for corresponding requests.
- Refreshing the current page with an HTTP request– The reload functionality re-sends an HTTP request for the current page and displays the updated content, title, and status code.
- Home page management: Loading default home page on startup and initialised with the university home page.
- Adding URLs to Favourites- Users can add URLs to a favourites list with a custom name. Also, can Modify and delete favourites.
- Loading favourites list and history list on startup and requesting the webpage by clicking the items.
- Buttons for navigating back and forth through the webpages.
- The application provides a Bulk Download feature where the user opens a file containing a list of URL's. The page then shows the URL, HTTP response status code associated with URL, and the number of bytes retrieved from this URL.
- Use of shortcut keys to increase accessibility.

List of requirements not implemented

- Creating and editing a home page URL.

DESIGN CONSIDERATIONS

In developing the web browser application, several key design decisions were made to ensure a robust, user-friendly, and efficient system.

Class Design

Form1: The main class, Form1, contains all the functionalities of the web browser application. Some of the main features include handling HTTP requests with `HttpClient`, navigation methods, error handling for HTTP status codes, and management of browsing history and favourites for user convenience.

Data Structures

- List Box: Utilized to display the history of visited URLs and the list of favourites, allowing users to interactively select and navigate to specific entries.
- Stack: Implemented for managing the back and forward navigation functionality. The stack enables easy tracking of previously visited pages, allowing users to move back and forth through their browsing history.
- List: Used for storing URLs in the favourites and history, providing a flexible structure for adding, modifying, and deleting entries.

HttpClient for HTTP Requests:

The HttpClient class is used for making HTTP requests, providing a modern and efficient way to handle network communications compared to older methods.

Modular Code Structure:

Functions like Load_HomePage, HttpError_handler, and Fetch_Content are designed for specific tasks, promoting code reuse and easier debugging.

File I/O Operations:

Manages user data persistence through file operations, allowing the application to save and load browsing history and favourites, enhancing user experience by retaining information between sessions.

Error Handling:

Robust error handling for various HTTP status codes, providing clear feedback and preventing application crashes.

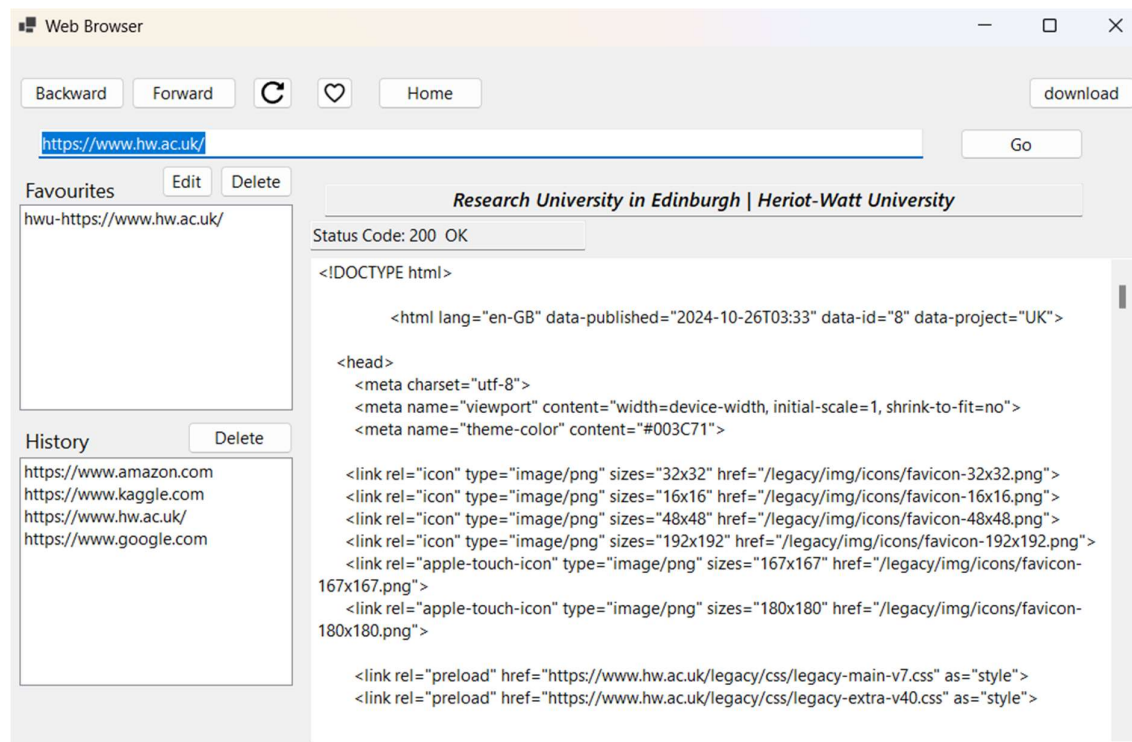
Asynchronous Operations:

Supports asynchronous URL loading and bulk downloads to keep the UI responsive during lengthy operations.

URL Verification:

The `verify_url` function ensures URLs are properly formatted by adding necessary prefixes, enhancing user experience and preventing errors during navigation and HTTP requests.

USER GUIDE



Home Page Initialization: The browser's home page is initialized to the university home page <https://www.hw.ac.uk/>.

Go Button (`go_button`)- Opens the URL entered in the URL text box , also performs while hitting enter to load a webpage.

Refresh Button (`refresh_button`)- Reloads the current page. Click this button to refresh the contents of the currently opened page.

Back Button (`backward_button`): Click this button to go back to the previously viewed page.

Forward Button (`forward_button`)-Goes forwards to the next page in browsing history.

Add to Favourites Button (`add_to_fav_button`)-Saves the current URL to the Favourites list.

Status Text Box (status_textbox)-Shows the HTTP response status of the last request.

History List Box (history_listBox)-It displays a list of URLs previously visited. Click on any of the URLs listed below to directly access that page.

Favourites List Box (fav_listBox)-Displays saved URLs marked as Favourites.

Bulk Download Button-Initiates the download process for URLs listed in a file.

History Management: Users can browse through their browsing history using the Back and Forward buttons to either view or jump to previously visited pages.

Favourites Management: User can add URLs to the favourites, edit or remove them, and can easily access these links.

Bulk Download Functionality: The user can open a file with many URLs to download in bulk and also see the HTTP status and response for each.

DEVELOPER GUIDE

Class Used

Form1: The main class of the application that contains core methods and GUI components, providing structure and complexity.

Home_button_Click loads the default home page (set to Heriot-Watt University's homepage).

Load_HomePage: Sets the homepage URL and initiates loading.

LoadUrl: Loads a URL asynchronously, showing the HTML content of a page and the HTTP status code.

DisplayContent: Updates the UI with the fetched HTML content and status code.

go_button_Click: Gets the URL from the textbox, validates its format and loads it.

backward_button_Click & forward_button_Click: Enables the user to navigate backward/forward through their browsing history by updating the back/forward stacks and loading the appropriate URL.

refresh_button_Click: Reloads the current page to refresh its content.

Home_button_Click: Navigate to home page URL defined in default.

verify_url: Verifies that the URL begins with "http" or "https"; if not, it prepends "https://".

add_to_fav_button_Click: Adds the current URL to the favorites list after user confirmation.

Load_Favourites: Loads favourites from a file and populates the listbox.

Edit_fav_button_Click: Allows editing of the selected favorite's name and URL.

Delete_fav_button_Click: Removes the current favorite from the list and the file.

Save_Favourites: Saves the modified favourites back to the file.

Add_History-Adds URL to browsing history and saves it to history file.

Load_History- Loads browsing history.

Bulk_download_Click-Opens a dialog for selecting a bulk download file and initiates downloads.

HttpError_handler :Displays error messages for various HTTP status codes.

TESTING

Various test cases were performed on the application.

- Tests to load URL
- Tests to navigate the page forward and back.
- Tests to refresh or reload the page.
- Tests to update history
- Tests to navigate the pages in history
- Tests to add the favourites, modify it and delete it
- Tests to show the output of Bulk Download
- Tests to check HTTP Error Messages
- Tests to check the shortcut keys

The tests to return the HTTP response status error code and outputs are the following.

1. Input URL: www.flipkart.com

Status Code: "403 Forbidden"

2. Input URL: www.httpbin.org/status/400

Status Code:"400 Bad Request"

3. Input URL: `www.example.com/thispagedoesnotexist`

Status Code: "404 Not Found"

REFLECTIONS ON PROGRAMMING LANGUAGE AND IMPLEMENTATION

This application asynchronous method that improves the user experience by using `async` and `await` to load content in the background without freezing the user interface. This made it possible to handle network-related tasks.

Listbox and Stack data structures were employed. Listbox assisted in saving and loading favourites and history data, while the Stack assisted in navigating the back and forward list.

The web browser was created using the Windows Forms App, which offers a user-friendly layout with a variety of buttons, textboxes, and listboxes.

The usage of files to store webpage URLs is one of the application's drawbacks; while files are helpful for small-scale applications, they may become problematic when a lot of web pages are loaded.

Other network problems, such as invalid URLs and permission concerns, are not reported by the program.

To overcome these limitations with the files we can use database solutions like SQL, MongoDB.

CONCLUSION

In Conclusion, this project effectively illustrates how to use Windows Forms to create a basic web browser application in *C#*. This browser offers a useful and effective surfing experience by combining core *C#* libraries, HTTP handling, and intuitive features like browsing history, favourites management, and bulk downloads. The project demonstrates the efficient use of *C#* features and best practices in software development, with a focus on code quality, secure coding, and contemporary design concepts. This project acts as a starting point for learning and investigating more complex *C#* web-related features and applications.

REFERENCES

Liberty, J. and MacDonald, B. (2008). *Learning C# 3.0*. 'O'Reilly Media, Inc.'

Ritscher, W. (2022). *Windows Forms - Advanced .NET: Commands, C# Enhancements, Web Apps, and Libraries*. [online] LinkedIn. Available at: <https://www.linkedin.com/learning/advanced-dot-net-commands-c-sharp-enhancements-web-apps-and-libraries/windows-forms?u=2374954> [Accessed 28 Oct. 2024].

w3schools (2022). *C# Tutorial (C Sharp)*. [online] www.w3schools.com. Available at: <https://www.w3schools.com/cs/index.php>.